

REST

REST (“Representational State Transfer”) é um estilo arquitetural amplamente utilizado para o desenvolvimento de APIs (Application Programming Interfaces) baseadas na web. Criado por Roy Fielding em sua tese de doutorado em 2000, o REST é projetado para tirar o máximo proveito do protocolo HTTP, fornecendo uma maneira simples e padronizada de comunicação entre sistemas.



Fonte: <https://pixabay.com/pt/illustrations/api-programas-interface-6492491/>

Uma API RESTful é caracterizada por ser leve, escalável e fácil de implementar. Suas principais características incluem:

- **Client-Server (Cliente-Servidor):** Separar as responsabilidades entre cliente e servidor permite maior flexibilidade e manutenção.
- **Stateless (Sem Estado):** Cada requisição enviada pelo cliente ao servidor deve conter todas as informações necessárias para processá-la. O servidor não armazena o estado da sessão.
- **Cacheability (Cacheável):** Respostas das APIs podem ser armazenadas em cache para melhorar a eficiência.
- **Interface Uniforme:** Uso consistente de recursos e operações que seguem padrões estabelecidos.

Vejamos alguns conceitos Fundamentais:

No REST, tudo é considerado um recurso; cada recurso é identificado por uma URL ("Uniform Resource Locator"). Por exemplo:

<https://api.exemplo.com/usuarios/1> — Representa o usuário com ID 1.

<https://api.exemplo.com/produtos> — Representa a coleção de todos os produtos.

Já os **verbos** HTTP são usados para realizar ações nos recursos. Os principais verbos são:

GET: Recupera informações sobre um recurso.

POST: Cria um novo recurso.

PUT: Atualiza completamente um recurso existente.

PATCH: Atualiza parcialmente um recurso.

DELETE: Remove um recurso.

Por exemplo:

GET /usuarios — Obtém uma lista de usuários.

POST /usuarios — Cria um novo usuário.

PUT /usuarios/1 — Atualiza as informações do usuário com ID 1.

DELETE /usuarios/1 — Exclui o usuário com ID 1.

Códigos de Status HTTP

As APIs RESTful usam códigos de status HTTP para indicar o resultado das operações:

- **200 OK:** Requisição bem-sucedida.
- **201 Created:** Recurso criado com sucesso.
- **204 No Content:** Operação bem-sucedida, sem corpo de resposta.

- **400 Bad Request:** Erro na requisição do cliente.
- **401 Unauthorized:** Autenticação necessária.
- **404 Not Found:** Recurso não encontrado.
- **500 Internal Server Error:** Erro no servidor.

O estilo arquitetural REST se tornou o padrão para desenvolvimento de APIs devido à sua simplicidade e flexibilidade. Entender seus conceitos básicos, boas práticas e como implementá-lo é essencial para criar aplicações web modernas, que sejam robustas e fáceis de manter.

Com a prática, você poderá criar APIs que suportem aplicações de alta performance e escalabilidade. Bons estudos!

GitHub da Disciplina

Você pode acessar o repositório da disciplina no GitHub a partir do seguinte link:

<https://github.com/FaculdadeDescomplica/Pratica-Integradora-Desenvolvimento-de-Apps>. Esse espaço é o seu portal para mergulhar fundo no universo da aprendizagem interativa. Nele, você encontrará todos os códigos, além dos links para os arquivos e dados.

Conteúdo Bônus

A partir do que foi apresentado neste módulo, recomenda-se ler o artigo abaixo.

Título: RESTful: Boas práticas para design de API

Autor: Marcello Gonçalves

Plataforma: [Dev.to](#)

Este artigo discute a importância de seguir boas práticas no design de APIs

RESTful para melhorar a segurança, escalabilidade e manutenção do serviço.

Referências Bibliográficas

BIESSEK, Alessandro. Flutter for Beginners. 1. ed. Birmingham: Packt Publishing, 2020.

SEJOURNÉ, Philippe. Flutter: Aplicações Nativas Multiplataforma com Flutter e Dart. 1. ed. Rio de Janeiro: Alta Books, 2020.

ZAMMETTI, Frank. Practical Flutter: Improve your Mobile Development with Google's Latest Open-Source Framework. 1. ed. Berkeley: Apress, 2019.

Atividade Prática 9 - REST

Título da Prática: Consumo de APIs REST em Aplicativos Móveis

Objetivos: Compreender como integrar e consumir APIs REST em aplicativos móveis para enviar e receber dados de servidores remotos.

Materiais, Métodos e Ferramentas: Para realizar esta prática, utilize o Flutter ou Android Studio (com Retrofit ou Dio) para consumir APIs REST. Ferramentas como Postman ou Insomnia são úteis para testar as APIs.

Roteiro

1º Passo) Leia atentamente o texto.

As APIs REST (Representational State Transfer) são amplamente utilizadas para permitir a comunicação entre um aplicativo cliente e um servidor remoto. Elas seguem princípios simples e são baseadas no protocolo HTTP. Uma API REST pode ter operações que realizam o CRUD (Criar, Ler, Atualizar e Deletar) de recursos, sendo um recurso qualquer objeto ou conjunto de dados manipulados por meio da API.

No desenvolvimento de aplicativos, você frequentemente interage com APIs REST para:

- **GET:** Obter dados do servidor.
- **POST:** Enviar dados para o servidor.
- **PUT/PATCH:** Atualizar dados no servidor.
- **DELETE:** Excluir dados no servidor.

As respostas dessas requisições geralmente estão no formato JSON, que é facilmente manipulado em linguagens de programação modernas.

2º Passo) Consuma uma API REST simples.

Imagine que você está criando um aplicativo que exibe uma lista de usuários. O servidor fornece uma API REST que retorna os dados dos usuários em formato JSON. A URL da API é <https://jsonplaceholder.typicode.com/users>.

O aplicativo deve consumir a API para obter e exibir a lista de usuários.

Exemplo de uma resposta da API em formato JSON:

json

CopiarEditar

```
[
{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "Sincere@april.biz"
},
{
  "id": 2,
  "name": "Ervin Howell",
  "username": "Antonette",
  "email": "Shanna@melissa.tv"
},
```

...
]

Utilize o Flutter (ou Android Studio) para realizar as seguintes ações:

1. **GET:** Faça uma requisição GET para obter os dados dos usuários.
2. **Mostrar os dados:** Exiba a lista de usuários na tela do aplicativo.
3. **Erro:** Se a requisição falhar, exiba uma mensagem de erro.

3º Passo) Responda às questões abaixo:

- A) Qual é o método HTTP utilizado para obter dados de uma API REST?
- B) Em uma API REST, qual operação é usada para enviar dados ao servidor?
- C) Qual é o formato mais comum utilizado nas respostas de uma API REST?
- D) Qual é o principal benefício de usar uma API REST em um aplicativo?

Gabarito Atividade Prática

- A) O método HTTP utilizado para obter dados de uma API REST é o **GET**, pois ele permite recuperar informações do servidor sem modificar os dados.
- B) A operação utilizada para enviar dados ao servidor em uma API REST é o **POST**, pois esse método permite criar novos registros ou enviar informações para processamento.
- C) O formato mais comum utilizado nas respostas de uma API REST é o **JSON**, pois é leve, fácil de ler e amplamente suportado por diversas linguagens de programação.
- D) O principal benefício de usar uma API REST em um aplicativo é **facilitar a comunicação entre o aplicativo e servidores remotos**, permitindo acesso a dados e funcionalidades externas de maneira eficiente.

Comentários e Conclusões

1. O que aprendeu com esta atividade?
2. Como o consumo de APIs REST pode melhorar a funcionalidade de um aplicativo?
3. Quais foram as dificuldades ao consumir a API e tratar os dados recebidos?